UFR 919 Sciences de l'ingénieur
Master Informatique – Parcours STL
Unité d'enseignement ALASCA

Gestion autonome d'énergie dans des logements « intelligents » autonomes

Jacques Malenfant
professeur des universités

© 2026. Ce travail est partagé sous licence CC BY-NC-ND.

version 1.0 du 23/09/2026

Résumé

L'objectif de ce projet est de comprendre l'utilisation des concepts, des approches et des techniques de modélisation, de simulation, d'implantation et de génie logiciel pour le développement des systèmes de contrôle cyber-physiques (autonomiques) et des jumeaux numériques. Cet objectif sera poursuivi par la réalisation d'une application visant à gérer automatiquement la production et la consommation d'énergie électrique d'un logement autonome doté d'équipements numérisés (appareils électriques, capteurs, panneaux solaires, éoliennes, batteries de stockage, ... équipés comme objets de l'internet). Ce projet se place dans le contexte de la transition énergétique par une évolution vers des logements et des quartiers écologiques utilisant essentiellement des énergies renouvelables en auto-consommation.

Rappel des dates et informations importantes (détails à la fin du document)

	Le projet ***doit*** être réalisé en ***Java SE 8***.
2/10/2026	Date limite pour la formation des équipes (2 personnes).
14/10/2026	Audit 1 (10% de la note finale).
22/11/2026	Rendu de code préalable à la soutenance de mi-semestre (format `tgz` ou `zip` *uniquement*).
25/11/2026	Soutenance de mi-semestre (semaine des ER1, 30% de la note finale).
6/01/2027	Audit 2 (10% de la note finale).
7/02/2027	Rendu de code préalable à la soutenance finale (format `tgz` ou `zip` *uniquement*).
10/02/2027	Soutenance finale (semaine des ER2, 50% de la note finale).
6-10/09/2027	Soutenance de seconde session (créneau à définir, note remplaçant entièrement celle de 1re session).

1 Problématique générale

Face à la pollution, aux changements climatiques et aux dangers non encore maîtrisés de l'énergie nucléaire (déchets, consommation d'eau, accidents, menaces de la guerre, ...) et malgré un potentiel développement de la fusion nucléaire toujours espéré, l'Union européenne ainsi que de nombreux autres acteurs publics et privés dans le monde ont engagé une double conversion de leurs réseaux de production et distribution d'électricité. Il s'agit :

— de réduire progressivement la part d'électricité produite par des grands équipements polluants (centrales au pétrole, au charbon ou au gaz, centrales nucléaires, ...) pour les remplacer par des unités de production fonctionnant à partir d'énergies renouvelables (soleil, vent, rivières/fleuves, marée, ...) ;
— de rendre « intelligents » (numériser) à la fois les appareils électriques ainsi que les équipements du réseau de production et distribution puis de les relier par réseau informatique

de manière à contrôler de manière beaucoup plus fine, précise et réactive l'équilibre entre production et consommation d'énergie.

Ces changements permettent de parler d'un passage aux *grilles d'énergie* dont le fonctionnement sera bien différent des réseaux de production et distribution d'aujourd'hui. En effet, un réseau électrique ne dispose pas (généralement) de stockage ; il doit donc absolument éviter la surcharge voire la disjonction du réseau en maintenant un niveau instantané de production de puissance électrique [1] supérieur à la consommation. Or, la production d'électricité via les énergies renouvelables est bien plus fluctuante et répartie sur le réseau que celle via les énergies fossiles ou nucléaires. Aujourd'hui, la production est très centralisée et les décisions pour ajouter (démarrage d'une centrale au gaz par exemple) ou retirer de la puissance produite se font sur des prévisions à plusieurs heures voire quelques jours, correspondant aux délais de mise en route et d'arrêt de tels équipements (facilement entre 8 et 24 heures pour des centrales au gaz, par exemple, plus encore pour les centrales au charbon). Ces décisions prises très en amont rendent la fluctuation dans la production invisible et pratiquement sans conséquence pour le consommateur.

Dans le cas des énergies renouvelables, des fluctuations importantes de la puissance produite apparaissent naturellement et ce, à des échelles de temps non plus de plusieurs heures mais de quelques *secondes* (vent fluctuant, temps partiellement nuageux, *etc.*). Ces fluctuations peuvent être amorties à court terme à l'aide d'unités de stockage d'énergie, mais l'équilibre entre puissance consommée et puissance produite doit absolument être retrouvé avant d'épuiser ces unités de stockage. Pour palier ces variations plus fortes et plus rapides, les grilles d'énergie prévoient que les consommateurs jouent un rôle central dans le maintien de l'équilibre entre production et consommation. Les trois principaux moyens envisagés côté consommateur pour assurer l'équilibre lorsque la consommation dépasse la production sur le réseau national sont :

1. Repousser certaines consommations à plus tard (resp. les avancer), par exemple en empêchant un appareil (chauffage, réfrigérateur, chauffe-eau, ...) de démarrer si la puissance produite est insuffisante, ou en reportant (resp. avançant) le démarrage d'appareils comme le lave-linge ou le four, surtout s'ils sont préprogrammés.
2. Utiliser des ressources locales, soit des unités de production (éoliennes, panneaux solaires, ...), soit des unités de stockage (batteries fixes du logement, batterie d'un véhicule électrique, ...) en auto-consommation pour compenser partiellement et/ou temporairement une production insuffisante du réseau. Les consommateurs disposant d'unités de stockage peuvent également acheter de l'énergie du réseau pour les recharger lorsque le réseau propose une énergie abondante et bon marché (par exemple, de l'énergie solaire pendant les journées à fort ensoleillement).
3. L'échange d'énergie directement entre consommateurs, certains possédant des unités de production ou de stockage dont ils peuvent temporairement céder une partie de la capacité de production.

Bien sûr, il n'est pas réellement envisageable que les consommateurs eux-mêmes prennent des décisions manuellement toute la journée, y compris la nuit, pour gérer leur consommation, leur production et leurs acquisitions d'énergie. L'automatisation de ces fonctions sera la clé d'une mise en œuvre réaliste de cette vision du futur. Une solution possible serait de confier à l'opérateur du réseau de distribution le soin de contrôler les appareils des logements via des compteurs « intelligents », mais cela demande de superviser les consommations au détriment de la protection de la vie privée.

Une alternative décentralisée consiste plutôt à doter chaque logement d'un contrôleur local, indépendant du compteur électrique, mais capable de « dialoguer » avec le réseau de distribution, de prendre puis d'appliquer des décisions au nom du consommateur à partir de règles de gestion décidées par ce dernier de manière opaque au réseau (pour protéger autant que possible les données personnelles des consommateurs). Notons dans les deux cas que la coopération des équipements et des appareils électriques est nécessaire, d'où le besoin d'évoluer vers des équipements et appareils numériques connectés souvent inclus dans ce que l'on désigne par « internet des objets ».

1. La puissance électrique, exprimée en watts ou en ampères (watts = ampères × voltage), est une mesure instantanée. Elle exprime soit une capacité produite par des générateurs soit une capacité consommée par des appareils électriques à un instant donné. L'énergie électrique est le produit d'une puissance par le temps pendant lequel cette puissance est fournie ou consommée. Elle s'exprime souvent en kilowatts∗heures.

CC BY NC ND

2 Gestion d'énergie « intelligente »

Mise en garde préalable : L'objectif du projet n'est pas le *réalisme* dans la matérialisation des phénomènes cyber-physiques (puissance électrique, évolution de température ambiante du logement ou interne au réfrigérateur, *etc.*) ou des contrôles exercés. Il s'agit principalement de comprendre globalement ces phénomènes et leurs implications. Par contre, une attention importante sera accordée à la bonne conception et programmation des composants et de l'architecture logicielle et à la qualité du développement, comme il se doit en développement logiciel, un des principaux métiers auxquels forme STL.

Aujourd'hui, de nombreuses offres commerciales existent qui équipent les logements de capteurs et permettent aux utilisateurs de connaître leur consommation d'énergie au fil du temps. À ce jour, ces offres visent simplement à faire prendre conscience à l'utilisateur sa consommation pour l'inciter à changer ses habitudes pour réduire le coût en énergie de son logement (et les émissions carbone). Notre objectif est d'aller beaucoup plus loin avec un gestionnaire d'énergie qui va prendre des décisions automatiquement avec deux objectifs :

— moduler les puissances produite et consommée pour assurer l'équilibre en tout temps ;
— si possible, réduire l'énergie totale consommée et donc le coût du logement en énergie.

Un objectif important d'un projet comme celui décrit dans ce cahier des charges serait de *dimensionner* la capacité totale des batteries et des unités de production à installer dans le logement pour éviter avec une forte probabilité (à fixer selon le client) de se voir privé d'électricité à un moment ou un autre étant donnée son profil de consommation prévisible. Pour cela, il faudrait procéder à des simulations sous conditions météorologiques (ensoleillement, vent, *etc.*) réalistes pour déterminer les bonnes capacités de production et de stockage, celles nécessaires et suffisantes pour éviter le surcoût d'équipement du logement (le coût des batteries et des unités de production est lourd, de fait).

2.1 Équipements intelligents

Pour arriver à automatiser la gestion d'énergie d'un logement, il faudra des équipements électriques dotés d'une capacité informatique et se prêtant à un contrôle à distance via réseau. [2] Ce genre d'appareils font partie de ce que l'on appelle l'Internet des objets. En effet, s'il est imaginable de contrôler la consommation électrique uniquement en pilotant le réseau électrique, [3] ce genre de pilotage en « tout ou rien » offre des possibilités limitées. Un appareil de chauffage dit « intelligent », par exemple, pourra offrir un contrôle beaucoup plus fin de sa consommation (en modulant sa puissance courante) que simplement être allumé ou forcé à être éteint en lui coupant le courant par la prise à laquelle il est connecté. Il permettra par exemple de fixer momentanément à distance sa puissance à une valeur entre 500 et 2000 watts.

2.1.1 Les appareils électriques

Dans l'esprit de l'internet des objets, projetons-nous donc dans un avenir pas si lointain où les équipements seront pilotables à distance par réseau et ce, en offrant une interface permettant de récupérer des informations sur ses fonctionnalités, sur son état courant ainsi que des opérations permettant de modifier son mode et ses paramètres de fonctionnement. Par exemple, un refrigérateur devrait pouvoir annoncer qu'il possède deux compartiments (froid et congélation), que chacun a une température cible pilotable à distance (généralement 4° pour la partie froid et -18° pour la partie congélation) ainsi que deux températures internes courantes pouvant être lues à distance et un ou deux compresseurs pouvant être mis en attente individuellement. Un autre exemple serait un chauffe-eau dont la puissance consommée peut être modulée à distance.

2. Il serait possible de se contenter d'équiper le réseau électrique interne du logement de prises contrôlables à distance (par exemple, des prises reliées en BlueTooth permettant de couper le courant en amont de l'appareil qui y est connecté), mais cela ne donnerait qu'un contrôle très grossier et pas toujours correct pour certains appareilss qui ne doivent pas être complètement débranchés.

3. Au delà des prises électriques contrôlables à distance, le même genre de dispositif peut se trouver au niveau du tableau électrique pour couper le courant circuit par circuit, sachant que certains appareils gros consommateurs comme les chauffe-eau sont généralement sur leur propre circuit.

CC BY NC ND

Interfaces propriétaires spécifiques versus interfaces génériques

Dans le contexte économique d'aujourd'hui, les nombreux fabricants d'appareils électriques offriraient vraisemblablement une myriade d'interfaces spécifiques à chacun de leurs appareils. Cela pose un défi majeur pour concevoir et réaliser un gestionnaire d'énergie *générique*, capable de piloter des appareils de différents fabricants. Une solution serait de normaliser les interfaces, mais il n'est pas toujours simple, même en accord avec tous les fabricants, de définir des normes uniques satisfaisantes. D'autant que le pilotage de la consommation d'énergie ne sera pas la seule application de la numérisation des appareils.

Pour mettre en œuvre un gestionnaire générique, il faut extraire les fonctionnalités essentielles de chaque appareil pour en piloter la consommation d'énergie. Dans ce but, nous allons classer les appareils électriques en deux catégories [4] :

1. Les appareils à consommation ajourd'hui *incontrôlable* et qui le resteront vraisemblablement vu leur faible coût, [5] dont on ne peut que subir les effets (sèche-cheveux, ventilateur, ...) selon la volonté de l'utilisateur.
2. Les appareils à consommation *modulables* qui sont souvent en opération permanente mais à consommation intermittente pouvant être suspendue ou modulée pour un temps plus ou moins long (refrigérateur, chauffe-eau, ...). Cette modulation de leur puissance consommée doit respecter des règles selon leurs conséquences (prévisibles) comme des règles de sécurité. [6]

Du point de vue du gestionnaire d'énergie, les appareils incontrôlables ne sont pris en compte que pour le total de la puissance qu'ils consomment à chaque instant, ce qu'il pourra déduire de la mesure globale de consommation électrique pour le logement obtenue du compteur électrique. Ces appareils ne sont ni connectés ni connectables. Par contre, pour être pilotés par le gestionnaire d'énergie, les appareils modulables vont lui être connectés. À cette fin, nous allons définir une interface *générique* permettant au gestionnaire de piloter n'importe quel appareil modulable, *sans en connaître la nature exacte*. Cette interface propose des opérations pour « suspendre » ou « reprendre » le fonctionnement nominal de l'appareil, chacun de ces derniers étant libre de décliner ce que suspendre et reprendre veut dire pour lui. Par exemple, pour tel réfrigérateur, suspendre empêche le compresseur de se mettre en route pour limiter temporairement sa consommation d'énergie. Pour tel chauffe-eau, suspendre est interprété comme réduire sa puissance à 10% de sa puissance nominale, par exemple. L'interface ajoute de la flexibilité à ce mécanisme, lorsque l'appareil le permet, avec un concept de modes de fonctionnement numérotés de 1 à n qu'il est possible de changer par des opérations comme *upMode*, *downMode* et *setMode*. Là encore, chaque appareil interprète cette notion de mode à sa guise, en définissant son nombre de modes et en attribuant une signification à chacun de ceux-ci. Par exemple, pour un appareil de chauffage à puissance modulable entre 500 et 2000 watts, l'interface permet de fixer des paliers (modes) à 500, 1000, 1500 et 2000 watts, numérotés de 1 à 4, entre lesquels l'appareil va passer par les opérations précédentes.

Bien sûr, pour chaque appareil, il faudra qu'une « traduction » s'opère entre ces opérations génériques appelées par le gestionnaire d'énergie et les opérations des interfaces propriétaires offertes par les appareils qui vont exécuter les changements de modalité de fonctionnement correspondantes. Par exemple, pour un réfrigérateur, l'opération *suspend* de l'interface générique devra appeler la méthode de son interface propriétaire suspendant son compresseur. Pour l'appareil de chauffage à puissance modulable, l'appel de l'opération *setMode* de l'interface générique pour fixer le mode 1 devrait appeler la méthode de son interface propriétaire fixant la puissance *setPower* en lui passant la valeur de puissance associée au palier minimal fixé précédemment à 500 watts. Nous revenons sur ce point un peu plus loin.

4. De manière plus réaliste, il faudrait proposer une série d'interfaces génériques dédiées à chaque type d'appareils (réfrigérateurs, appareils de chauffage, fours, lave-linge, etc.). De fait, une troisième catégorie d'appareils *planifiables* regroupe ceux qui disposent d'une fonctionnalité de programmation comme le départ différé sur les lave-linge et les lave-vaisselle. Pour simplifier le projet, nous ne traitons pas cette catégorie pour ne conserver qu'une seule catégorie d'appareils modulables.

5. Ajouter un micro-processeur et une interface réseau de type Bluetooth sur un sèche-cheveux restera vraisembvlablement prohibitif vu le coût de ce genre d'appareils.

6. Par exemple, la température dans le réfrigérateur ne doit pas monter trop haut pour une trop longue durée sinon on prendrait des risques sanitaires ou de perte sur les aliments.

CC BY NC ND

Comme indiqué auparavant, le test et le dimensionnement des équipements pour un logement exige de pouvoir simuler les puissances produite et consommée, lesquelles dépendent souvent des conditions du monde physique, comme la température extérieure du logement qui influence la puissance et l'énergie consommées pour le chauffage. La complexité de la modélisation des appareils électriques courants varie passablement d'un appareil à l'autre. L'annexe A présente une liste d'appareils classés en fonction de la difficulté de les modéliser et de les simuler. Cette classification distingue :

- les appareils « simples », notés de type "S" par la suite, dont la puissance consommée évolue uniquement en fonction des actions de l'utilisateur (allumer, éteindre, augmenter ou baisser la puissance, etc.) ;
- les appareils « à consommation variable », notés de type "V" par la suite, dont la puissance et l'énergie consommées sont influencées par les évolutions de phénomènes du monde physique nécessitant une simulation numérique par équation différentielle, comme un réfrigérateur ;
- les appareils « à consommation variable automatique », notés de type "A" par la suite, qui ajoutent aux appareils de type "V" le fait qu'ils modifient leur puissance consommée de manière automatique, soit parce qu'ils fonctionnent par « programmes » (par exemple, un lave-linge dont la puissance consommée dépend de la phase du programme – lavage, rinçage, essorage, etc. – et entre lesquelles il passe de manière automatique), ou qu'ils ont une durée de fonctionnement programmable qui arrête leur consommation de manière automatique (par exemple, un four à minuterie qui s'arrête lorsque le temps de cuisson prévu est écoulé ou un appareil de chauffage programmable selon les périodes de la journée) [7]

Dans le cadre du projet, vous devrez inclure [8] :

1. ***Au moins un appareil incontrôlable***.
2. ***Au moins un appareil de type "S"***.
3. ***Au moins un appareil de type "V"***.
4. ***Au moins un appareil de type "A"***.

Si vous le souhaitez, un appareil de type "A" (en plus de celui exigé) peut remplacer un appareil de type "V" ou "S" et un appareil de type "V" (en plus de celui exigé) peut remplacer un appareil de type "S".

2.1.2 Les unités de production

De manière similaire, les unités locales de production d'énergie peuvent être classées dans deux catégories :

1. Les unités de production *aléatoires* (panneaux solaires, éoliennes, ...) qui peuvent produire en permanence mais dont on subit la fluctuation dans le niveau de production en fonction des aléas de l'environnement (ensoleillement, vent, ...) ; ces unités offrent des possibilités d'intervenir uniquement pour des raisons de sécurité ou d'entretien (arrêt des éoliennes pour leur mise en sécurité lorsque le vent est trop fort, par exemple).
2. Les unités de production *intermittentes* (turbine hydro-électrique à bassin de rétention [9],

7. Un départ différé ou à une heure précise sont aussi des fonctionnalités intéressantes qui pourraient être utilisées par le gestionnaire d'énergie pour décaler le départ afin de faire face à une baisse momentanée de la puissance produite au sein du logement. Toutefois, comme expliqué auparavant, pour simplifier le projet l'interface générique de supervision des appareils modulables ne tient pas compte de cette fonctionnalité. Il s'agit donc ici de comprendre comment modéliser et implanter de tels appareils, même si dans la suite du projet, le gestionnaire d'énergie ne pourra pas moduler les heures de départ de ces appareils.

8. Tout au long du projet, des exemples vous serons fournis pour vous aider à mieux comprendre ce qu'il faut faire et ce qui est attendu. Dès le départ, vous aurez donc des exemples d'appareils et un début de programmation du projet. Vous devrez inclure ces éléments fournis dans votre projet, mais les éléments demandés ici ***s'ajoutent*** aux éléments fournis que vous choisirez d'inclure dans votre projet.

9. Une turbine hydro-électrique à bassin de rétention possède un bassin de rétention d'eau en amont grâce auquel elle produit en laissant s'écouler l'eau jusqu'à épuisement (niveau minimal du bassin atteint) ; la recharge de ce bassin se fera par apport naturel d'eau via la pluie. On pourrait aussi envisager d'installer une turbine hydro-électrique continue sur un cours d'eau, s'il en existe un à proximité, qui produirait un niveau de puissance à peu près stable, dans la durée. Dans le cadre du projet, cela n'apporterait pas grand chose car cela fixerait simplement un seuil minimal de production sur lequel le gestionnaire d'énergie pourrait toujours compter. Nous excluons donc arbitrairement cette possibilité.

CC BY NC ND

génératrices à essence ou à gaz) qui ne produisent pas en permanence mais qui peuvent « à coup sûr » (sauf pannes, ce qu'on va exclure) produire une puissance électrique dont le niveau sera possiblement modulable, pour une durée bornée et prévisible (par exemple, selon le niveau d'eau du bassin ou des réservoirs de gaz ou d'essence).

Dans le cadre du projet, des unités de production vous seront fournies avec des interfaces permettant de les piloter. Lors de la dernière étape du projet, votre gestionnaire d'énergie devra les intégrer dans sa stratégie de gestion. Ces unités seront intégrées dans l'ensemble du système via le compteur électrique. La simulation de leur puissance produite sera ainsi intégrée à la simulation globale du logement.

Cas particulier : unités de stockage

Les unités de stockage se distinguent par deux vues complémentaires : une vue « fournisseur d'énergie » (déchargement des batteries pour fournir de l'énergie, par exemple [10]), qui se comporte comme une unité de production intermittente (selon le niveau d'énergie restant dans les batteries, par exemple), et une vue « consommateur d'énergie » (chargement des batteries, par exemple) qui se comporte comme un appareil consommant de la puissance électrique. L'unité de stockage sera également connectée au gestionnaire d'énergie via une interface permettant de la piloter à distance.

Dans le cadre du projet, pour être plus autonome, le logement sera doté de batteries. En tant qu'unité de production intermittente, les batteries ont cette restriction que contrairement à une génératrice à essence par exemple, elles ne se « démarrent » pas. Les batteries sont en permanence connectées au compteur électrique et elles sont *automatiquement* sollicitées *dès* que le niveau de puissance produite passe sous le niveau de la puissance consommée, sans qu'il soit nécessaire de les déclencher explicitement. Cela évite que le réseau électrique disjoncte à cause d'une consommation plus forte que la production [11]. Ce comportement est tout à fait similaire à ce qui se passe dans votre ordinateur portable où la batterie prend instantanément le relais du secteur quand on le débranche de la prise de courant. Lorsque les batteries passeront en mode production d'énergie, l'objectif du gestionnaire d'énergie sera de réduire au plus vite et le plus possible la puissance totale consommée dans le logement pour éviter d'épuiser complètement les batteries avant de pouvoir les recharger.

En tant qu'appareil consommant de l'énergie lors de la charge, il faudra traiter les batteries comme un *appareil modulable*, c'est à dire que le chargement des batteries est possible en permanence mais ne se fait que de manière intermittente lorsque nécessaire [12], comme le compresseur d'un réfrigérateur est disponible en permanence mais fonctionne (et consomme de la puissance) uniquement pendant que la température interne du réfrigérateur demeure au-dessus d'une certaine limite. Le gestionnaire d'énergie peut suspendre le chargement des batteries si la situation exige plutôt de faire fonctionner d'autres appareils modulables et que la puissance produite par les unités de production est insuffisante. Toutefois, en tant qu'appareil modulable, les batteries ne seront pas gérées *via* l'interface générique discutée plus haut car le gestionnaire d'énergie doit prendre des décisions spécifiques qui imposent de savoir qu'il opère sur des batteries et non sur un appareil modulable quelconque.

10. Une turbine hydro-électrique à bassin de rétention peut aussi disposer d'un bassin bas pour retenir l'eau à partir duquel elle peut se « recharger » en pompant l'eau du bas vers le haut, ce qui consommera de l'énergie en période de production autre forte. Une telle turbine serait alors plutôt vue comme une unité de stockage. Pour le projet, nous utiliserons simplement des batteries.

11. Il reste possible que le réseau disjoncte si la consommation excède le total de la production et de la puissance maximale de sortie des batteries (courant de décharge nominal). Les batteries tolèrent généralement un excès en fournissant une puissance bien plus élevée (courant de crête) mais sur une période courte (quelques dizaines de secondes) au risque d'endommager ou de limiter sévèrement leur durée de vie.

12. Le chargement des batteries n'est bien sûr activé que lorsque la puissance produite par les unités de production, hors batteries, dépasse la puissance consommée, puisqu'on ne saurait utiliser la puissance produite par les batteries pour les charger elles-mêmes.

CC BY NC ND

2.1.3 Le compteur électrique

De plus, il sera nécessaire de pouvoir obtenir les niveaux totaux instantanés de puissances électriques consommée et produite du logement. Pour plus de réalisme, cela sera assuré par un compteur électrique numérique. Un composant compteur vous sera fourni dès le début du projet. Aux étapes 2 et 3, il faudra y intégrer la simulation de la puissance consommée par vos appareils. Comme les unités de production et de stockage, le compteur électrique sera connecté au gestionnaire d'énergie pour permettre à ce dernier de récupérer les niveaux de puissances produite et consommée.

2.1.4 Le gestionnaire d'énergie

Le gestionnaire d'énergie est l'élément central du système car c'est lui qui rend le service demandé, à savoir le pilotage de la production, de la consommation et du stockage d'énergie afin d'assurer l'équilibre en tout temps. Le gestionnaire est d'abord un contrôleur au sens de ce que nous voyons en cours. À travers les connexions spécifiques au compteur électrique ainsi qu'aux unités de production et de stockage de même qu'aux connexions génériques avec les appareils modulables, il perçoit l'état courant du système et agit pour assurer l'équilibre à tout instant. Les connexions fournissent en réalité les capteurs et les actionneurs, bien que dans son cas nous n'ayons pas fait apparaître des interfaces capteurs et actionneurs explicites, plus conformes aux architectures cyber-physiques vues en cours.

Mais pour assurer au mieux sa fonction, il est possible d'élargir les perspectives et de le voir plus généralement comme un *jumeau numérique* du système. En tant que jumeau numérique, il accumule des données d'historique sur la production et la consommation d'énergie afin :

— de produire une visualisation de l'état du système au fil du temps,
— de piloter les unités de production et de stockage ainsi que les appareils modulables grâce à des prévisions de production et consommation à court terme,
— d'analyser le comportement du système à plus long terme pour faire du dimensionnement (plutôt en phase de conception, par simulation MIL) et en améliorer le pilotage (plutôt dynamiquement, en détectant des écarts de modèles et en refaisant des simulations avec les nouveaux modèles).

Pour ne pas surcharger le projet, nous n'allons pas aller très loin dans l'implantation d'un jumeau numérique en nous concentrant uniquement sur les fonctions contrôleur et prédictions à court terme. Pour cela, nous allons faire quelques hypothèses supplémentaires réalistes :

— que les appareils modulables disposent de capteurs internes de consommation d'énergie dont les valeurs sont accessibles par une opération de l'interface générique de pilotage des appareils modulables ;
— que les unités de production fournissent également dans leurs interfaces spécifiques des opérations permettant de connaître leur niveau de production instantanée et d'autres informations pertinentes (par exemple, la consommation de carburant ainsi que le niveau de carburant restant dans son réservoir pour un générateur) ;
— que le logement dispose d'une station météo connectée à laquelle le gestionnaire d'énergie aurait accès pour obtenir des données de température extérieure, de température intérieure et d'ensoleillement.

Grâce à ces informations, le gestionnaire d'énergie va pouvoir construire des prévisions à court terme de production et de consommation d'énergie sur lesquelles il va se baser pour prendre ses décisions. Conceptuellement, on peut imaginer qu'un jumeau numérique comme celui-ci pourrait développer des modèles très sophistiqués et à long terme. En pratique, pour le projet, on va se contenter de produire des analyses très simples à court terme (quelques minutes à quelques heures par rapport à l'instant courant).

2.2 Gestion de l'énergie et contrôle

Si nous pouvions déployer une telle application dans un logement *réel*, nous pourrions connaître la puissance électrique consommée par les différents appareils en plaçant des capteurs de mesure sur le réseau ou grâce à des capteurs intégrés aux appareils et interrogeables à distance. Malheu-

CC BY NC ND

reusement, nous ne disposons pas d'une plate-forme expérimentale pour cela, et ce serait de toutes façons actuellement un effort trop important dans le cadre d'un cours de 6 ECTS. Nous allons donc nous contenter d'un développement logiciel *sans matériel* où la *simulation remplacera les mesures réelles.* [13]

Côté logiciel, un appareil électrique numérique est doté d'un logiciel embarqué et il est connecté via réseau, ce qui permet d'échanger des informations et de piloter ses fonctions à distance. Dans le projet, chaque appareil, chaque unité de production et unité de stockage sera représenté par un composant logiciel dont les services et les méthodes internes formeront le logiciel qui serait embarqué (déployé) sur l'appareil correspondant. Les connexions (via ports et connecteurs) entre composants vont abstraire (et remplacer ici) le fait que celles-ci puissent être établies via réseau sans fil de type WiFi ou Bluetooth dans la réalité.

Pour développer du logiciel assez réaliste, nous allons également nous tourner vers la simulation numérique. Chaque appareil, chaque unité de production et chaque unité de stockage devra être à même de simuler sa puissance électrique consommée ou produite (en ampères ou en watts). Pour les appareils simples et incontrôlables, il suffit souvent d'avoir une puissance constante : soit l'appareil est allumé et il consomme une puissance fixe (par exemple, un sèche-cheveux consommant 1200 watts), soit il est éteint et il ne consomme bien sûr rien. Un cas à peine plus complexe est un appareil incontrôlable ou modulable qui a différents modes de fonctionnement (par exemple, un ventilateur à trois niveaux de puissance). Dans ces cas, le simulateur produira une consommation électrique constante par morceaux et le basculement entre les niveaux se fera par des actions représentées par des événements dans la simulation : allumage, arrêt, passage de mode de bas à haut et vice versa, *etc.*

Pour les appareils incontrôlables, ces actions sont déclenchées par les utilisateurs des appareils, à simuler comme des événements. Pour les appareils modulables, en plus des actions de l'utilisateur (allumer et éteindre, par exemple), des actions sont déclenchées par le gestionnaire d'énergie (suspendre et reprendre, par exemple), ce qui exige de déclencher des événements dans la simulation pour provoquer les changement de niveaux de consommation. Grâce aux simulateurs, le compteur électrique pourra « mesurer » les niveaux totaux de puissances consommée et produite dans le logement. Le composant compteur électrique offrira des services capteurs permettant au gestionnaire d'énergie de récupérer ces valeurs pour prendre ses décisions et les actions en fonction de la situation (écart entre production et consommation, niveau des batteries, *etc.*).

D'autres appareils, plus complexes, en plus des actions de l'utilisateur ou de pilotage à distance, passent d'un mode à l'autre de manière *autonome*. Ces changements de modes peuvent se produire à cause d'une limite physique. Par exemple, la capacité des batteries étant limitée, elles passent du mode « en charge » à « chargé » lorsque cette limite est atteinte. Pour d'autres appareils, ces changements de mode se produisent par une *action* interne qui peut être *prédéterminée* ou de *contrôle*. Voici quelques exemples d'actions automatiques dans des appareils de type "A" :

— Lave-linge : la fin de la phase de rinçage fait passer à la phase essorage.
— Four : l'expiration de la durée de cuisson arrête le four de chauffer.

Voici quelques exemples d'appareils de types "V" ou "A" qui intègrent une fonction de contrôle simple :

— Chauffage : à partir d'une température cible et d'une lecture de la température ambiante dans le logement, le thermostat du radiateur allume et éteint les éléments chauffants de telle manière à maintenir la température ambiante proche de la température cible.
— Chauffe-eau : de manière similaire à celle d'un appareil de chauffage, le chauffe-eau maintient une température cible de l'eau dans son réservoir en agissant sur des éléments de chauffage électriques placés dans le réservoir.
— Réfrigérateur : il agit de même pour maintenir la température cible de ses compartiments, à ceci près qu'il utilise un compresseur pour refroidir et non des éléments de chauffage pour augmenter la température ; un réfrigérateur à un compresseur refroidit ses deux compartiments (s'il y en a deux) en même temps alors qu'avec deux compresseurs, les deux compartiments sont gérés indépendamment.

13. En fait, nous allons voir en cours que de toutes façons, le développement d'une version co-simulée avec le logiciel, dîte en simulation « *software-in-the-loop* », précède généralement celui d'une version simulée pour l'environnement uniquement mais incluant le matériel dîte « *hardware-in-the-loop* », avant le passage en déploiement réel.

CC BY NC ND

— Lave-linge : durant la phase de lavage, lorsque la température de l'eau atteint le niveau requis par le programme (par exemple, 30° pour le programme « délicat »), le lave-linge commence ses rotations et contre-rotations pour laver effectivement le linge.

Pour ces contrôleurs intégrés aux appareils, il faut disposer de capteurs et d'actionneurs internes puis implanter un algorithme de contrôle comme vu en cours. Le plus simple sera d'utiliser des formes de contrôle basiques venant de l'automatique comme les contrôleurs à seuils avec *hystérésis* que nous verrons en cours, mais il vous sera possible d'utiliser des contrôleurs plus complexes si vous le souhaitez.

Au-dessus de ces actions et contrôles internes, le gestionnaire d'énergie du logement va exercer lui aussi une forme de contrôle à partir de la collecte de données sur les puissances totales instantanées produites et consommées ainsi que de l'état courant des appareils et de prédictions les consommations et productions futures. Il conservera une liste des appareils dont la consommation a été modulée (réfrigérateur, chauffe-eau, chauffage, ...). À partir de ces informations, il pourra décider des actions de gestion d'énergie appropriées. Ce type de contrôle pourra utiliser les techniques présentées en cours dans le contexte de l'informatique autonomique ou de la robotique autonome.

2.3 Influence de l'environnement

L'environnement inclut tous les facteurs extérieurs influençant la gestion d'énergie en dehors des équipements (hormis les actions des utilisateurs). Par exemple, on va intégrer, le cas échéant :

— la température extérieure qui influence via la porosité thermique des murs du logement la manière dont décroît la température intérieure ;
— la température de l'eau froide provenant du réseau de distribution d'eau qui influe sur la température de l'eau du réservoir d'eau chaude lorsque la consommation d'eau chaude nécessite son remplissage ;
— l'intensité d'ensoleillement courant qui influe sur la puissance produite par des panneaux solaires ;
— la force du vent qui influe sur la puissance produite par des éoliennes.

Les éléments entrant en ligne de compte pour la gestion de l'énergie dans votre projet, selon vos choix d'appareils, doivent être intégrés dans l'architecture logicielle par des capteurs « connectés » permettant de connaître les valeurs courantes de ces facteurs environnementaux (thermomètre extérieur, thermomètre sur le compteur d'eau, anémomètre, ...). Pour être en mesure de mettre au point, de tester systématiquement et de dimensionner le système, il faudra aussi simuler ces phénomènes pour produire des données d'entrée réalistes.

2.4 Impact des utilisateurs

Dans un logement, les activités des utilisateurs sont à l'origine de la demande d'énergie des appareils. Par exemple, les phénomènes suivants pourront être intégrés :

— les choix des températures cibles pour les appareils concernés ;
— les moments et durée de consommation d'eau chaude (aux heures de repas pour laver la vaiselle, matin pour les douches, soir pour les bains, ...) ;
— les accès au refrigérateur (ouvertures et fermetures des portes) qui influent sur la rapidité de l'augmentation de la température interne des compartiments ;
— allumage et extinction des appareils à consommation incontrôlable.

Encore ici, pour mettre au point, tester le système et le dimensionner, il faudra simuler ces comportements, selon les appareils qui seront inclus dans le système. Toutefois, le comportement des utilisateurs peut différer passablement. Il sera bon de prévoir des paramètres de simulation permettant de faire varier les scénarios utilisateurs afin de dimensionner en fonction de ceux-ci (nombre de personnes vivant dans le logement, par exemple).

CC BY NC ND

3 Mise en œuvre du projet

Plan général

Le projet va se dérouler en quatre étapes correspondant aux quatre évaluations prévues :

Étape 1 (3 semaines, du 23/09 au 14/10) : Développement en BCM4Java des composants représentants les appareils que vous aurez choisis, sans toutefois tenir compte de la puissance électrique qui sera intégrée aux étapes suivantes via la simulation.

Étape 2 (6 semaines, du 14/10 au 25/11) : Développement des simulateurs modulaires permettant de simuler les puissances électriques consommées et produites ainsi que les autres éléments influençant celles-ci : températures internes, facteurs environnementaux, actions des utilisateurs, *etc.*

Étape 3 (6 semaines, du 25/11 au 6/01/2027) : Intégration des simulateurs modulaires développés à l'étape 2 au sein des composants développés à l'étape 1 pour former des composants cyber-physiquesde la co-simulation « *software-in-the-loop* ».

Étape 4 (5 semaines, du 6/01 au 10/02) : Développement du gestionnaire d'énergie comme jumeau numérique ainsi que des scénarios de tests d'intégration et de dimensionnement du système.

Étape 1 — Composants équipements et leurs interconnexions

***Nota bene* :** À cette étape, nous nous intéressons uniquement à la partie composants logiciels du projet, sans simulation, et donc en ne tenant aucun compte de l'électricité. [14] Ce n'est qu'à l'étape 2 que nous aborderons la partie purement électrique, c'est à dire la production et la consommation de puissance électrique en termes quantitatifs, par la simulation. De même, les divers contrôles (internes aux appareils, comme la température du four, ou sur la consommation globale de la maison dans le gestionnaire d'énergie) ne seront pour leur part programmés qu'à l'étape 3, lorsque les simulateurs seront intégrés aux composants. Toutefois, les méthodes nécessaires devront être incluses dans les interfaces et des « bouchons » pour les tests inclus dans le code.

Pour cette première étape, chaque équipement est représenté par un composant avec au moins une interface offerte, que vous devez concevoir, explicitant les opérations permises sur celui-ci selon les entités tierces les utilisant (gestionnaire d'énergie, actions des utilisateurs, actions du contrôleur local selon les appareils, *etc.*). Le compteur électrique et le gestionnaire d'énergie, également représentés par des composants, vous sont fournis. [15]

Tous les appareils électriques modulables, les unités de production et l'unité de stockage seront connectés (par ports et connecteurs BCM4Java) au gestionnaire d'énergie pour être pilotés à distance par ce dernier. Le rôle du composant représentant le compteur électrique ne sera effectif qu'à l'étape 3 quand des données de puissances électriques consommées et produites seront obtenues par la simulation et que les simulateurs auront été intégrés aux composants. À cette étape 3, les mesures des puissances totales produite et consommée faites par le composant compteur depuis son simulateur pourront être récupérées par le gestionnaire via des méthodes dont les signatures vous sont fournies.

Une fois que vous aurez sélectionné les appareils que vous allez inclure dans votre projet, il faudra faire une conception logicielle (composant) de chacun puis les intégrer avec les composants fournis. Pour ce faire, il faudra déterminer les opérations qui seront disponibles sur ces derniers et pour quels types de composants tiers de manière à spécifier les interfaces qu'ils vont offrir. Vous devrez ensuite implanter les différents composants puis produire un assemblage de composants exécutable avec des scénarios de test.

14. Au sens des simulateurs DEVS vus en cours ; pour les besoins des tests unitaires ou des tests d'intégration, il sera possible de simuler certains éléments comme le temps courant pour s'assurer du bon fonctionnement du code sans avoir à attendre des heures pour le déroulement d'un test en temps réel. Voir les exemples fournis pour ce genre de « simulations temporelles » pour le test logiciel.

15. Le composant compteur ainsi qu'une version préliminaire du composant gestionnaire vous seront fournies. Dans la suite du projet, de nouvelles versions du compteur électrique vous seront aussi fournies. Pour le gestionnaire d'énergie, il faudra le modifier pour inclure le code nécessaire pour compléter votre projet au fil du semestre.

CC BY NC ND

Connexion BCM4Java générique des équipements à consommation modulable

***Nota bene* :** La problématique de connexion *générique* des appareils au gestionnaire d'énergie attaquée dans cette sous-section concerne en réalité tous les appareils, unités de production et unités de stockage. Toutefois, pour ce qui vous concerne, vous n'avez à traiter de manière générique que les appareils modulables. [16] Les appareils incontrôlables ne sont pas concernés, puisqu'ils ne sont pas connectés au gestionnaire d'énergie. Pour les autres équipements, ils sont connectés au gestionnaire d'énergie par des interfaces de composants spécifiques fournies.

Dans la description générale en introduction, nous avons évoqué la problématique pragmatique de la connexion (informatique) des équipements avec le gestionnaire d'énergie. En BCM4Java, il est possible de connecter deux composants (ici le composant client gestionnaire d'énergie et un composant serveur représentant un équipement électrique, comme un réfrigérateur) le premier requérant une interface A et l'autre offrant une interface B différente de A. Il faut alors implanter un connecteur qui est appelé selon l'interface A par le composant client mais qui appelle le composant serveur selon l'interface B, faisant ainsi la « médiation » ou la « colle » entre les deux.

Dans une économie de marché libre, il est difficile d'imaginer que tous les équipements électriques proposés par tous les industriels au monde se conforment aux mêmes interfaces standards. Mais si chaque équipement électrique produit dans le monde devant pouvoir être connecté à notre gestionnaire d'énergie offrait une interface spécifique propriétaire, il faudrait prévoir à l'avance tous les connecteurs pour tous ces équipements et des appels côté composant gestionnaire spécifiques à chacune de ces interfaces, une solution ingérable en pratique, d'autant que de nouveaux appareils nécessitant de nouveaux connecteurs apparaîtraient tout le temps pendant la durée de vie du gestionnaire d'énergie. Cette problématique très courante dans l'industrie va nous permettre d'explorer les possibilités offertes par la *génération de code à l'exécution* à l'aide de l'outil Javassist pour les appareils contrôlables.

Dans le cadre du projet, nous supposons que le gestionnaire d'énergie requiert (au sens de BCM4Java) une interface de contrôle *générique* via laquelle il va piloter les appareils modulables appelée `AdjustableCI` (qui vous est fournie). Pour connecter un appareil modulable offrant une interface de composant spécifique, `RefrigeratorCI` par exemple, il faut un connecteur qui fait la médiation entre l'interface requise `AdjustableCI` et l'interface offerte `RefrigeratorCI`.

Dans le code fourni comme point de départ du projet, ces connecteurs sont programmés manuellement et vous les trouverez dans le répertoire du composant gestionnaire. Vous pourrez procéder de la même manière dans un premier temps pour vos propres appareils. Mais cette solution est insatisfaisante, comme expliqué ci-haut. Dans un second temps, vous devrez passer à de la génération de code des connecteurs nécessaires *à la volée*. Pour savoir comment implanter les méthodes définies dans l'interface requise (par exemple, `AdjustableCI`) en termes d'appels à l'interface offerte (par exemple, `RefrigeratorCI`), on va utiliser un descripteur en XML fourni par l'équipement. [17] Cette approche vous permettra d'expérimenter des architectures logicielles complexes, où on cherche à connecter tardivement (*pendant l'exécution*) des parties de logiciels développés avec différentes interfaces voire dans différents langages de programmation, ce qui devient de plus en plus courant dans l'industrie avec ce qu'on appelle les *systèmes de systèmes*.

Maintenant, comment un appareil va fournir au composant gestionnaire d'énergie son descripteur XML ? Dans le cadre du projet, le composant gestionnaire d'énergie va offrir une interface d'enregistrement et de connexion pour les appareils modulables appelée `RegistrationCI`. Selon cette approche, lors de leur enregistrement, les appareils vont fournir au gestionnaire leur document XML. Ce document XML décrit par du code source Java comment chaque méthode de l'interface requise générique va être implantée pour appeler les méthodes de l'interface spécifique offerte par l'appareil. Ce document XML suit un schéma Relax NG fourni (`control-adapter.rnc`). Il doit être utilisé pour générer à la volée le code de la classe de connecteur à utiliser pour connecter (au sens de BCM4Java) le composant gestionnaire au composant représentant l'appareil qui souhaite s'enregistrer.

16. Dans les exemples qui vous sont fournis comme point de départ du projet, les appareils de ce type (appareil de chauffage) sont connectés via des connecteurs programmés *manuellement*. Vous aurez à faire en sorte dans votre livraison de modifier cela pour remplacer la connexion par connecteur généré automatiquement.
17. Un exemple complet vous sera fourni.

CC BY NC ND

Voici en résumé le scénario qui doit être implanté. Tous les appareils connaissent l'URI du port du gestionnaire d'énergie offrant l'interface d'enregistrement `RegistrationCI`. Pour être géré par celui-ci, un composant implantant un appareil modulable qui vient d'être allumé s'y connecte et appelle la méthode `register` pour s'enregistrer. Cette méthode prend un identifiant unique de l'équipement (son numéro de série, par exemple), l'URI de son port entrant offrant son interface de contrôle et le document XML décrivant son interface de contrôle. Le gestionnaire va alors générer dynamiquement une classe de connecteur avec Javassist puis l'utiliser pour se connecter à l'appareil. [18] Lorsque l'équipement est éteint, il se désenregistre du gestionnaire en appelant la méthode `unregister`.

Modalité de réalisation

À titre de suggestion, la réalisation de cette étape peut elle-même se décomposer en tâches :

— identification des équipements à inclure dans votre projet (appareils et unités de production) ;
— recherches personnelles et sur Internet pour trouver des informations réalistes sur les fonctionnalités des équipements sélectionnés pour bien modéliser leurs services ;
— définition des interfaces de composants puis développement des composants (les puissances électriques n'étant pas modélisées à cette étape, contentez-vous de vous assurer que vos équipements passent bien d'un mode à l'autre quand les opérations sont appelées et que les opérations permettant de récupérer des données fonctionnent correctement) ;
— programmation manuelle des connecteurs pour les équipements qui ne sont pas concernés par la connexion générique (appareils incontrôlables, unités de production, batteries) ;
— programmation manuelle des connecteurs pour les composants représentant vos appareils modulables (cela vous facilitera ensuite la production du descripteur en XML) ;
— programmation du générateur de connecteurs en utilisant l'outil Javassist et développement des documants XML pour vos appareils modulables à partir du code Java des connecteurs développé manuellement à l'étape précédente ;
— assemblage des composants pour former une application exécutable ;
— programmation de scénarios de démonstration en utilisant les méthodes `start` et `execute` des composants qui va enregistrer les appareils auprès du gestionnaire puis tester les fonctionnalités sur les appareils via ce composant gestionnaire (comme démontré dans le logiciel fourni comme point de départ).

Pour la génération du code des connecteurs à partir d'un document XML, vous pouvez vous référer à la classe `fr.sorbonne_u.components.cvm.config.ConfigurationFileParser` de BCM4Java pour un exemple de lecture et traitement de fichiers XML en Java.

Objectifs à atteindre

À l'issue de cette étape, lors de l'audit 1, vos réalisations devraient permettre :
— d'expliquer votre conception, les choix que vous aurez faits et l'état d'avancement que vous aurez atteint ;
— d'avoir les composants appareils demandés avec leurs tests unitaires spécifié dans le langage Gherkin [19] et exécutables individuellement comme illustré avec les deux exemples de composants appareils fournis (le sèche-cheveux et l'appareil de chauffage) ;
— d'avoir intégré vos composants appareils avec les versions fournies des composants compteur et gestionnaire d'énergie, lequel sera complété pour exécuter les tests, dans un assemblage exécutable ;
— de réaliser la connexion des appareils modulables via le protocole d'enregistrement auprès du gestionnaire d'énergie incluant la génération du code des connecteurs au moment de

18. Dans votre développement, il est vivement conseillé de programmer d'abord manuellement les connecteurs de vos appareils et d'utiliser ces connecteurs manuels pour mettre au point votre code. Seulement ensuite, vous vous attaquerez à la génération automatique, pour bien séparer les préoccupations en vous concentrant alors sur la génération de code.
19. À l'URL : `https://cucumber.io/docs/gherkin/reference`, vous trouverez un petit manuel de référence de ce langage.

BY NC ND

l'enregistrement avec Javassist ;
— d'exécuter des scénarios de testsunitaires et d'intégration (que vous aurez bien sûr spécifiés et programmés) vérifiant le bon fonctionnement du code des appareils en suivant les changements de mode de fonctionnement ordonnés par le gestionnaire et prévus dans les scénarios ;
— de faire cette démonstration en mono-JVM (exécution des scénarios de tests).

Étape 2 — Simulation MIL

À venir.

Étape 3 — Intégration en simulation SIL et contrôles locaux

À venir.

Étape 4 — Gestionnaire d'énergie et scénarios d'exécution

À venir.

4 Modalités de remise et d'évaluation des projets :

— Le projet se fait **obligatoirement** en **équipe de deux étudiant(e)s**. Tous les fichiers sources du projet doivent comporter les noms (balise `author`) de tous les auteurs en Javadoc. Lors de sa formation, chaque équipe devra se donner un nom d'équipe et me le transmettre avec les noms des étudiant(e)s la formant au plus tard le **2 octobre 2026** à minuit.
— Le projet doit **impérativement** être exécutable sous **Java SE 8** ! Attention, peu importe la JVM et le système d'exploitation sur lequel vous travaillez, il faudra que votre projet s'exécute correctement sous Eclipse en Mac Os X (que j'utilise, donc attention aux variabilités si vous utilisez un autre système d'exploitation).
— Votre rendu *final* de projet devra inclure une *documentation Javadoc* des différents paquetages de votre projet. La documentation Javadoc sera générée et incluse dans votre livraison dans un répertoire `doc` au même niveau que votre répertoire `src`.
— Votre code final doit aussi répondre des standards de qualité attendus à votre niveau (Bac+5 prêts à partir en stage en entreprise), être commenté, lisible et correctement présenté (indentation, choix pertinents des identifiants, ...).
— L'évaluation comportera quatre épreuves : deux audits intermédiaires, une soutenance à mi-semestre accompagnée d'un rendu de code et une soutenance finale accompagnée du rendu final du code du projet et de la documentation associée. Les deux audits intermédiaires dureront 20 minutes chacun. La soutenance à mi-parcours durera 30 minutes. La soutenance finale durera 45 minutes. Le tout se déroulera selon les modalités suivantes :

1. Les **deux audits intermédiaires** auront lieu pendant les séances du cours, *a priori* le **14 octobre 2026** (séance 4) puis le **6 janvier 2027** (séance 11). Ils se dérouleront de manière informelle devant votre ordinateur où vous pourrez me présenter l'état d'avancement de votre projet et répondre à mes questions. Ils donneront lieu à une note sur 20 valant chacune 10% de la note finale de l'UE. L'audit 1 visera à vérifier que vous avez atteint les objectifs de l'étape 1. L'audit 2 portera sur l'atteinte des objectifs de l'étape 3 tel que décrits précédemment.
2. La **soutenance à mi-parcours** portera sur l'atteinte des objectifs des deux premières étapes du projet et elle aura lieu dans la semaine des premiers examens répartis, *a priori* le **25 novembre 2026**. Elle donnera lieu à une note sur 20 comptant pour 30% de la note finale de l'UE. Elle comportera une discussion des réalisations (code) pendant une quinzaine de minutes (devant l'écran sous Eclipse) et une courte démonstration de cinq minutes. Les rendus à mi-parcours se feront le **dimanche 22 novembre 2026 à minuit** au plus tard.

CC BY NC ND

3. La **soutenance finale** portant sur l'ensemble du projet mais avec un accent sur les troisième et quatrième étapes aura lieu dans la semaine des seconds examens répartis, *a priori* le **10 février 2027**. Elle donnera lieu à une note sur 20 comptant pour 50% de la note finale de l'UE. Elle comportera une présentation d'une douzaine de minutes (en utilisant quelques transparents) insistant sur les scénarios de tests développés et les résultats obtenus, une discussion d'une dizaine de minutes également devant écran sur les réalisations suivie d'une démonstration (scénarios de tests d'intégration et scénarios de dimensionnement). Les rendus finaux se feront le **dimanche 7 février 2027 à minuit** au plus tard.

— Bien que les audits et les soutenances se font par équipe, l'évaluation reste à chaque fois **individuelle**. Lors des audits et des soutenances, *chaque étudiant(e)* devra se montrer capable d'expliquer différentes parties du projet, et selon la qualité de ses explications et de ses réponses, sa note peut être supérieure, égale ou inférieure à celle de l'autre membre de son équipe.

— Les audits et les soutenances auront lieu selon un horaire de passage qui sera publié le moment venu sur le site de l'UE. **Tout retard** de l'équipe de plus de la moitié de la durée prévue de ces audits ou soutenances entraînera une **absence** à l'épreuve et une note de 0 sera attribuée aux deux membres de l'équipe pour l'épreuve concernée. Si un(e) seul(e) des membres d'une équipe arrive après l'heure limite, il(elle) sera absent(e) de l'épreuve et une note de 0 sera attribuée. Dans ce cas, si l'autre membre de l'équipe est à l'heure, il(elle) passera l'épreuve seul(e).

— Le rendu à mi-parcours et le rendu final se font sous la forme d'une archive contenant un répertoire directement désarchivable sous un répertoire de projet Eclipse. Le répertoire et votre archive ***doivent*** avoir pour nom celui de votre équipe (ex. : l'équipe LionDeBelfort utilise un répertoire `LionDeBelfort` et l'archive dans `LionDeBelfort.tgz` ou `LionDeBelfort.zip`). Cette archive doit se présenter au format `tgz` si vous travaillez sous Unix ou `zip` si vous travaillez sous Windows et ce ***à l'exclusion de tout autre format !*** Vous m'enverrez le fichier à `Jacques.Malenfant@lip6.fr` comme attachement fait proprement avec votre programme de gestion de courrier préféré ou encore par téléchargement depuis un site tiers avec un lien envoyé par courrier électronique (en lieu et place du fichier).

  Pour éviter de transmettre des archives trop volumineuses qui seront rejetées par le système de courrier électronique, ne transmettez que le strict minimum, essentiellement les répertoires de sources Java de votre projet. Évitez d'inclure les binaires et les jars qui vous sont fournis, par exemple. Par contre, si vous utilisez des bibliothèques au format jar autres que celles fournies, alors il faudra les inclure dans la livraison.

— **Tout manquement à ces règles élémentaires entraînera une pénalité dans la note des épreuves concernées !**

— Pour l'épreuve de deuxième session, si elle s'avérait nécessaire, elle se tiendrait dans la semaine du 6 au 10 septembre 2027, sur un créneau de soutenance qui sera annoncé au besoin en temps et en heure avec un rendu de code et de documentation le dimanche 5 septembre à minuit au plus tard. Le travail à réaliser consiste à poursuivre le développement du projet pour résoudre ses insuffisances constatées à la première session.
  Quand une seule personne dans l'équipe doit passer la seconde session, elle fait ce travail et passe l'épreuve seule ; l'évaluation tient alors compte du fait que le rendu de seconde session a été fait par une seule personne. Une personne peut également demander (en avance, dès les résultats de la première session connus) à passer la seconde session seul(e) même si les deux membres de l'équipe doivent se présenter à l'épreuve ; dans ce cas, les deux personnes travaillent et passent l'épreuve en solitaire selon les modalités précédentes.

CC BY NC ND

A Modélisation et simulation des appareils électriques

A.1 Appareils modulables

L'un des objectifs centraux d'ALASCA est de montrer comment la simulation du monde physique est nécessaire pour faire du test des systèmes cyber-physiques bout-en-bout réalistes. Par contre, cette simulation peut inclure une quantité variable de phénomènes physiques et informatiques à prendre en compte. Nous classons ici les appareils électriques courants en fonction de la difficulté de leur modélisation et de leur simulation.

A.1.1 Appareils « simples »

Les appareils modulables « simples » ont un comportement en termes de puissance consommée dépendant de leur mode de fonctionnement, lui-même résultant uniquement d'actions de l'utilisateur qui peuvent aussi être effectuées à distance pour un appareil modulable :

Lampe à variateur : lampes à forte consommation (typiquement halogène) dont on peut fixer la puissance maximale consommée, le plus souvent par un variateur. Mais pour simplifier le projet, on peut supposer pour ce type d'appareil à variateur qu'il existe des modes de fonctionnement correspondant à des paliers fixées par avance à 25, 50, 100, ... watts plutôt qu'une plage continue de variation.

Ordinateur multi-cœur avec accélérateur graphique : ordinateur dont on peut imaginer qu'il soit pilotable à distance pour activer/désactiver certaines de ses capacités comme le nombre de cœurs utilisables ainsi que leurs fréquences ou l'utilisation ou non de l'accélérateur graphique. En tant qu'appareils « simples », on considère que ces modes de fonctionnement s'imposent à toutes les applications en cours d'exécution et correspondent à des paliers de puissance consommée connus à l'avance.

Box internet : comme pour l'ordinateur, appareil dont certaines fonctions (réseau WiFi, espace de stockage, etc.) peuvent être activées/désactivées à distance pour obtenir des paliers de puissance consommée connus à l'avance.

Ventilateur : ventilateur haut de gamme à plusieurs niveaux de puissance qui peuvent être pilotés à distance (comme cela se fait avec une télécommande).

D'autres appareils peuvent être ajoutés, par exemple en considérant que certains appareils qui sont incontrôlables dans le bas de gamme puissent être contrôlables en version haut de gamme, comme l'exemple du ventilateur.

A.1.2 Appareils « à consommation variable »

Dans le contexte du projet, les appareils modulables à consommation variable sont des appareils similaires à l'appareil de chauffage fourni en exemple, dont on peut moduler la puissance maximale consommée en leur imposant à distance un mode de fonctionnement. Mais, contrairement aux appareils « simples », leur puissance consommée dépend aussi de conditions plus complexes à simuler parce que faisant intervenir des comportements continus venant de l'environnement ou internes, modélisés par équations différentielles et des changements de mode par action de contrôle (comme lorsqu'on atteint la température cible pour l'appareil de chauffage) :

Réfrigérateur : un réfrigérateur dont on peut moduler la puissance maximale consommé en suspendant momentanément le fonctionnement du compresseur pour un temps limité. Mais la puissance consommée par le compresseur dépend de la température interne en s'activant et se désactivant selon que le réfrigérateur a atteint leur température cible ou non.

Congélateur, cave à vin, climatiseur, pompe à chaleur : tous ces appareils partagent un comportement assez similaire soit au réfrigérateur soit à l'appareil de chauffage modulo parfois des particularités spécifiques.

Chauffe-eau : un chauffe-eau permet de fixer la température cible de l'eau et, dans le cadre du projet, de moduler la puissance maximale consommée. L'évolution de la température de l'eau impose l'utilisation d'une simulation numérique de la température de l'eau. L'évolution

CC BY NC ND

de la puissance consommée est compliquée par le fait que la consommation d'eau chaude dépend du comportement des habitants du logement et que la température de l'eau froide en entrée du réservoir varie en fonction de la période de la journée et des saisons.

Bouilloire pilotable : une bouilloire haut de gamme dont la température cible peut être fixée à distance, dont l'arrêt est automatique et qui possède une fonction de maintien au chaud. L'arrêt et le maintien au chaud dépend de la température cible et de celle de l'eau dont l'évolution suit une équation différentielle. Dans le contexte du projet, il faut ajouter la possibilité de moduler la puissance maximale consommée par palier.

Déshumidificateur : appareil qui utilise souvent un ventilateur interne et des filtres pour extraire l'humidité de l'atmosphère ; sa puissance consommée dépend du taux d'humidité cible et de celui de la pièce qui évolue selon une équation différentielle également, un peu comme un appareil de chauffage mais où l'évolution du taux d'humidité remplace celle de la température.

A.1.3 Appareils « à consommation variable automatique »

Les appareils contrôlés « à consommation variable automatique » ajoutent aux appareils « à consommation variable » une complexité supplémentaire par le fait qu'ils ont des changements de modes automatiques déclenchés par le temps ou par des événements externes, ce qui rend leur modélisation et leur simulation un peu plus exigentes :

Four : un four programmable intègre d'abord un comportement assez similaire à celui du chauffe-eau. Ici, la température de la pièce joue un rôle assez similaire à celui de la température de l'eau froide pour le chauffe-eau mais avec une évolution différente.[20] S'ajoutent la complexité de la durée de cuisson programmable. Une possibilité de modulation après le début de la cuisson serait de borner la puissance, comme pour l'appareil de chauffage, ce qui ralentirait la remontée en température mais n'empêcherait pas la cuisson. Par contre, il paraîtrait plutôt imprudent d'autoriser la suspension du four pendant la cuisson, les effets pouvant être très mauvais sur le résultat…

Lave-linge : un lave-linge opère selon différents programmes dont les phases (chauffage de l'eau jusqu'à la température requise par le programme, durée des phases dont le lavage, intensité de l'essorage, etc.) ont des puissances maximales consommées variables. Le chauffage de l'eau impose les mêmes complexités que le chauffe-eau mais il ajoute la multiplicité des programmes et la modulation des quantités d'eau en fonction de la charge. On peut aussi imaginer qu'un programme puisse être suspendu momentanément pour un temps limité.

Lave-vaisselle : assez similaire au lave-linge.

Serveur informatique élastique : un serveur informatique diffère de l'ordinateur (considéré comme appareil « simple ») par le fait qu'il peut exercer un contrôle d'admission sur les tâches à exécuter. Il peut inhiber ou retarder l'exécution de certaines tâches selon les capacités qu'elles exigent pour réduire la puissance consommée ; généralement celles qui sont gourmandes en puissance électrique comme l'accès à l'accélérateur graphique, l'utilisation du réseau ou de baies de stockage. On peut en quelque sorte comparer le flôt de tâches soumises au flôt d'eau froide dans le réservoir d'un chauffe-eau, à ceci près que les exigences des tâches sont plus variables. Pour tenir compte de cela, il impose :

— de simuler les exigences de chaque tâche, éventuellement en besoins instantanés maximaux ainsi qu'en variation continue au cours de l'exécution ;
— de modéliser le flôt de tâches soumises, avec des mélanges de différents types de tâches en termes d'exigences.

En comparaison avec l'ordinateur (appareil « simple »), en plus de la possibilité de fixer des limites momentanées à ses capacités, le serveur peut choisir les tâches à exécuter ou à retarder pour diminuer sa puissance consommée instantanée plutôt que de limiter arbitrairement cette dernière sans tenir compte des besoins, ce qui rend sa simulation plus compliquée. Pour ce type d'appareils, dans le cadre du projet, il faut savoir quel degré de réalisme est nécessaire car cela peut aller très loin.

20. Par simplicité, il n'est pas obligatorie de considérer l'effet du four qui fonctionne sur la température de la pièce, qu'il fait monter bien sûr dans la réalité.

CC BY NC ND

B Scénarios de tests et horloge accélérée

Le test des applications avec interfaces graphiques et encore plus des applications parallèles et réparties exige de pouvoir planifier des actions de différentes entités logicielles à des instants précis les unes par rapport aux autres pour construire des scénarios d'exécution corrects et réalistes. Cela demande donc de synchroniser ces entités de telle manière à ce qu'elles exécutent les actions dans un ordre précis. Plusieurs techniques sont envisageables pour ce faire, dont celle d'une synchronisation *dirigée par le temps* que nous allons utiliser dans le cadre du projet CPS.

B.1 Scénarios de test temporisés

Pour le projet, par exemple, vous comprendrez vite que lors du démarrage, il faut que les composants nœuds du réseau de capteurs s'inscrivent auprès du registre (action 1) avant que les composants clients puissent récupérer auprès du registre (action 2) les informations de connexion sur ces nœuds pour leur envoyer des requêtes (action 3). Si on ne synchronise pas ces actions, des conditions de course (*race conditions*) vont prévaloir entre les composants et selon l'ordre dans lequel la machine virtuelle Java va exécuter les choses, on pourra très bien avoir l'action 2 qui sera exécutée avant l'action 1, ce qui mènera à une erreur.

BCM4Java offre une classe appelée `AcceleratedClock` ainsi qu'un composant `ClocksServer` pour simplifier la construction de scénarios de test temporisés. Premièrement, pour synchroniser les actions des composants de manière dirigée par le temps, il faut que tous les composants puissent accéder à une unique horloge qui va établir un temps global suffisamment précis pour qu'on puisse faire exécuter une série d'actions par différents composants dans un ordre choisi par le programmeur. Que l'on soit sur un seul ordinateur ou sur plusieurs ordinateurs dont les horloges matérielles sont synchronisées avec suffisamment de précision, l'horloge matérielle est une bonne candidate pour servir de temps global.

En Java, la méthode `System.currentTimeMillis()` retourne la valeur de l'horloge matérielle de l'ordinateur sous-jacent exprimée en ce qui est appelé l'*Unix epoch time*, c'est à dire en temps écoulé depuis le 01/01/1970 en millisecondes. Par ailleurs, les *pools* de *threads* ordonnançables de l'`ExecutorService` de Java offrent la possibilité de faire exécuter du code après un certain délai. BCM4Java s'appuie sur cette implantation pour offrir la méthode `scheduleTask` :

```
final AbstractComponent c = this;
this.scheduleTask(
  o -> { c.traceMessage("Ceci sera exécuté après le délai d nanosecondes.\n"); },
  d, TimeUnit.NANOSECONDS);
```

Attention, pour pouvoir utiliser la méthode `scheduleTask` (et les autres méthodes du même genre), il faut que le composant possède au moins un *thread* dans un *pool* de *threads* ordonnançable.

B.2 Horloge accélérée

Bien que l'horloge matérielle donne une base fixe et fiable sur plusieurs ordinateurs, planifier des actions de test dans cette référence temporelle n'est pas aisée. Pour faciliter la planification des actions, la classe `AcceleratedClock` de BCM4Java offre la possibilité d'utiliser une autre référence temporelle pour planifier les actions : celle de la classe `Instant` de Java. Sans entrer dans tous les détails, il est facile de créer un instant « virtuel » grâce à la méthode statique `Instant#parse` qui prend en paramètre une chaîne de caractères où un instant est exprimé selon la norme internationale suivante :

"2025-01-31T09:00:00.00Z"

c'est à dire année-mois-jour puis heure, minutes, secondes et centièmes de secondes, le Z indiquant le fuseau horaire. L'expression `Instant.parse("2024-01-31T09:00:00.00Z")` retourne donc l'objet instant représentant le 31 janvier 2025 à 9 heures du matin.

CC BY NC ND

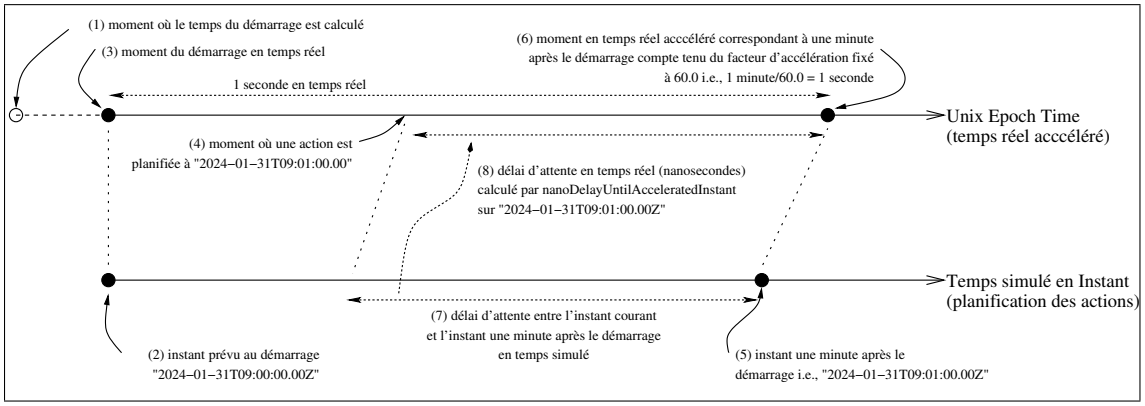


FIGURE 1 – Les deux références temporelles mises en synchronisation par `AcceleratedClock` : le temps réel accéléré s'appuyant sur l'horloge matérielle de l'ordinateur via le système d'exploitation et le temps virtuel des actions du scénario de test représenté par des `Instant` de Java.

Supposons que l'instant précédent représente l'instant de démarrage des actions du scénario de test, grâce aux instants Java on peut planifier une action devant se dérouler une minute après le démarrage à 9h01. Soit on crée un nouvel instant avec :

```
Instant.parse("2024-01-31T09:01:00.00Z")
```

Soit on crée de manière équivalente un nouvel instant par rapport à un instant précédent :

```
Instant i0 = Instant.parse("2025-01-31T09:00:00.00Z");
Instant i1 = i0.plusSeconds(60);
```

On constate qu'il sera beaucoup plus facile et lisible de travailler avec des instants Java pour planifier les actions d'un scénario de test qu'avec l'*Unix epoch time* en millisecondes.

Maintenant, la question qui se pose est comment mettre en relation un temps simulé exprimé en `Instant` avec le temps Unix qui sert de base pour ordonnancer l'exécution de tâches à des moments précis en temps réel. L'idée va être de créer une horloge qui va maintenir une synchronisation entre les deux références temporelles. Cette classe, `AcceleratedClock`, va permettre de créer une horloge en lui passant un moment précis en temps réel Unix pour le démarrage des actions du scénario de test disons `t0` et un instant en temps simulé du scénario exprimé en `Instant` disons `i0`. Après le démarrage du scénario (après `t0`), l'horloge va permettre de convertir des temps réel Unix (postérieurs à `t0`) en instants de même que des instants (postérieurs à `i0`) en temps réel Unix.

Un autre aspect intéressant à prendre en compte est la possibilité de planifier des actions selon une échelle de temps commode mais de pouvoir ensuite exécuter le scénario beaucoup plus rapidement. Par exemple, on pourrait trouver plus simple de planifier un scénario de test avec des actions se déclenchant toutes les minutes, mais ensuite d'accélérer ce temps pour faire en sorte qu'une minute planifiée s'exécute en une seconde en temps réel. Pour ce faire, `AcceleratedClock` va permettre de définir à la création d'une horloge un facteur d'accélération entre le déroulement du temps simulé et le déroulement du temps réel.

Pour comprendre, considérez la figure 1. En (1), on va calculer et programmer le temps de démarrage du scénario de test. Souvent, ce sera fait dès le début de l'exécution de l'application et il faudra laisser suffisamment de temps pour l'initialisation et le démarrage de tous les composants. En définissant un délai initial de démarrage, on pourrait définir le moment de démarrage de la manière suivante :

```
public static final long START_DELAY = 3000L; // 3000 millisecondes
long unixEpochStartTimeInNanos =
  TimeUnit.MILLISECONDS.toNanos(System.currentTimeMillis() + START_DELAY);
```

CC BY NC ND

En (2), apparaît l'instant de départ matérialisé sur la référence temporelle du scénario de test et en (3) le moment du démarrage en temps réel est matérialisé sur la référence temporelle du temps réel Unix.

Pour pouvoir se référer à une horloge, on lui attribue à sa création une URI. Ainsi, la création d'une horloge accélérée peut se faire de la manière suivante :

```
public static final String CLOCK_URI = "horloge-101";
AcceleratedClock ac =
  new AcceleratedClock(
      CLOCK_URI,        // URI attribuée à l'horloge
      t0,               // moment du démarrage en temps réel Unix
      i0,               // instant de démarrage du scénario
      60.0)             // facteur d'acccélération
```

Avant de pouvoir faire des calculs avec cette horloge, il faut attendre le moment du démarrage en temps réel. Pour cela, la classe `AcceleratedClock` définit une méthode `waitUntilStart()` qui, comme son nom l'indique bloque le *thread* qui l'appelle jusqu'à ce que l'on arrive en temps réel au temps `t0` choisi pour le démarrage du scénario.

Supposons maintenant qu'à un certain moment dans l'exécution (voir (4) sur la figure) se situant avant le moment en temps réel où elle doit être déclenchée, le programme planifie une action devant se déclencher à l'instant `i1`, c'est à dire une minute après le démarrage du scénario de test qui est matérialisé en (5). Grâce à la synchronisation modulo le facteur d'accélération, `AcceleratedClock` peut reporter cet instant `i1` sur la référence temporelle du temps réel Unix en (6). Elle peut alors calculer le délai en termes d'instants à attendre depuis le moment où on veut planifier cette action jusqu'à `i1` en (7). Elle peut aussi calculer le même délai mais cette fois-ci en temps réel Unix, indiqué en (8). Ce délai en temps réel est celui qu'il faut utiliser dans la méthode `scheduleTask` pour faire s'exécuter l'action au bon moment :

```
long d = ac.nanoDelayUntilInstant(i1); // délai en nanosecondes
final AbstractComponent c = this;
this.scheduleTask(
  o -> { c.traceMessage("Le code de l'action 1 doit être placé ici.\n"); },
  d, TimeUnit.NANOSECONDS);
```

Le moment où la planification est faite est le moment auquel l'expression `ac.nanoDelayUntilInstant(i1)` est exécutée et le délai `d` est le délai en temps réel Unix à attendre avant de déclencher l'exécution de l'action 1.

B.3 Partage d'horloges accélérées : le serveur d'horloge

Maintenant que l'on a vu les principales fonctionnalités de l'horloge accélérée, [21] se pose un nouveau problème : comment faire en sorte que plusieurs composants, s'exécutant potentiellement sur plusieurs ordinateurs, puissent partager une *même* horloge accélérée afin que leurs actions planifiées soient effectivement synchronisées selon le temps qui s'écoule sur cette horloge.

Comme on l'a répété en cours, on ne peut pas partager une référence Java à un objet horloge entre plusieurs machines virtuelles Java. La solution consiste à fournir un composant serveur d'horloge sur lequel on va créer des horloges avec leurs URIs puis les composants vont pouvoir appeler ce composant pour récupérer des copies de cette horloge en utilisant l'interface `ClocksServerCI` offerte par `ClocksServer` et requise par les composants qui souhaitent l'utiliser. Toutes les copies d'une même horloge seront synchronisées entre elles par le fait qu'elles utilisent l'horloge matérielle comme référence temporelle. Que ces composants soient tous sur le même ordinateur et donc se réfèrent à la même horloge matérielle par définition offrant une synchronisation parfaite, ou qu'ils soient sur différents ordinateurs donc les horloges matérielles de ces ordinateurs seront supposées suffisamment synchronisées entre elles pour que les horloges accélérées le soient également.

21. Consultez la documentation javadoc de BCM4Java pour plus de détails.

CC BY NC ND

Dans le cadre du projet, il suffira de créer un composant serveur d'horloge à partir de la classe `ClocksServer`. Ce composant permet de créer une horloge en passant les paramètres nécessaires à l'un de ses constructeurs. Par exemple, pour l'horloge précédente, cela donne :

```
public static final String TEST_CLOCK_URI = "test-clock";
public static final Instant START_INSTANT =
                                Instant.parse("2024-01-31T09:00:00.00Z");
protected static final long START_DELAY = 3000L;
public static final double ACCELERATION_FACTOR = 60.0;

long unixEpochStartTimeInNanos =
  TimeUnit.MILLISECONDS.toNanos(System.currentTimeMillis() + START_DELAY);

AbstractComponent.createComponent(
  ClocksServer.class.getCanonicalName(),
  new Object[]{
        TEST_CLOCK_URI,             // URI attribuée à l'horloge
        unixEpochStartTimeInNanos,  // moment du démarrage en temps réel Unix
        START_INSTANT,              // instant de démarrage du scénario
        ACCELERATION_FACTOR});      // facteur d'acccélération
```

Ensuite, chaque composant qui veut se synchroniser sur cette horloge va devoir se connecter au serveur d'horloge, récupérer une copie de l'horloge en utilisant son URI puis l'utiliser pour faire sa planification :

```
ClocksServerOutboundPort p = new ClocksServerOutboundPort(this);
p.publishPort();
this.doPortConnection(
        p.getPortURI(),
        ClocksServer.STANDARD_INBOUNDPORT_URI,
        ClocksServerConnector.class.getCanonicalName());
AcceleratedClock ac = p.getClock(TEST_CLOCK_URI);
this.doPortDisconnection(p.getPortURI());
p.unpublishPort();
p.destroyPort();
// toujours faire waitUntilStart avant d'utiliser l'horloge pour
// calculer des moments et instants
ac.waitUntilStart();
```

B.4 Précautions à prendre

La précision de l'ordonnancement sur un ordinateur utilisant un système d'exploitation comme Unix n'est pas très élevée. Par expérience sur mon Unix, la précision est d'environ 10 millisecondes en pratique. Cela veut dire que si deux composants appellent `scheduletask` au même moment avec des délais différant de moins de 10 millisecondes, on ne peut plus prévoir dans quel ordre les deux tâches seront exécutées.

En pratique, cela veut dire que le délai en temps réel séparant deux tâches à ordonnancer ne doit pas être inférieur à 10 millisecondes pour obtenir l'ordre que l'on souhaite dans un scénario. Et bien sûr, il faut prendre en compte le facteur d'accélération qui aboutit à réduire les délais en temps réel entre les actions planifiées à deux instants. Il faut donc éviter de produire des scénarios où le délai entre deux actions selon la référence temporelle des instants divisé par le facteur d'accélération ne donne une valeur inférieure à 10 millisecondes. Par exemple, deux actions planifiées à une minute d'intervalle dans le temps simulé avec un facteur d'accélération de 600 vont se produire à un dixème de secondes de distance (60 secondes/600) en temps réel, donc ne devrait pas poser de problème. Par contre, avec un facteur d'accélération de 6000, on tomberait à une différence de 10 millisecondes, donc pouvant potentiellement poser des problèmes.

CC BY NC ND

Pour votre culture personnelle, sachez qu'il existe des horloges matérielles et des systèmes d'exploitation beaucoup plus précis dans l'ordonnancement des tâches, c'est à dire pouvant descendre au millionnième voire au milliardième de seconde. Ce sont des systèmes d'exploitation temps réel utilisés dans les applications temps réel.

Fin du document.

CC BY NC ND